

BROTAR

Plataforma web de crowdfunding

DEFINICIÓN DEL STACK TECNOLÓGICO Y DIRECTRICES DE ARQUITECTURA

Estándares oficiales para los equipos de desarrollo

Documento de referencia para Frontend, Backend y Base de Datos. Su propósito es que todos los equipos trabajen con las mismas tecnologías y criterios de organización.

1. Propósito del documento

Este documento establece el stack tecnológico y las directrices de arquitectura que deberán seguir los equipos que desarrollen Brotar. La intención no es añadir complejidad innecesaria, sino mantener una base común para que el código sea entendible, ordenado y fácil de mantener.

Como criterio general, se utilizará TypeScript tanto en frontend como en backend. Esto permite trabajar con tipado estático en toda la aplicación, reducir errores comunes y mantener estructuras de datos más claras entre la interfaz web y la API.

Enfoque para el MVP. Las arquitecturas definidas deben aplicarse de forma práctica. No es necesario crear capas, abstracciones o patrones que no aporten valor al problema actual. Primero debe mantenerse el código claro y correctamente separado.

2. Resumen del stack oficial

Capa	Tecnología	Lenguaje / herramienta	Arquitectura
Frontend	React	TypeScript + Bun	Screaming Architecture / Feature-First
Backend	NestJS	TypeScript + pnpm	Arquitectura Hexagonal
Base de datos	PostgreSQL	SQL	Modelo relacional
Acceso a datos	Prisma o TypeORM	TypeScript	Solo dentro de Infrastructure

Regla importante. El frontend utilizará Bun para su gestión de dependencias y scripts. El backend utilizará pnpm. No se deben mezclar gestores dentro de la misma parte del proyecto.

3. Frontend

3.1 React + TypeScript

La interfaz web se desarrollará con React y TypeScript. No se utilizará JavaScript plano para el código principal del frontend. TypeScript permitirá tipar propiedades de componentes, formularios, modelos y respuestas de API.

- Los componentes deben tener propiedades (props) tipadas.
- Los datos recibidos o utilizados por una funcionalidad deben tener tipos o interfaces claras.
- Se debe evitar el uso indiscriminado de any.
- Los errores del compilador de TypeScript deben corregirse y no ignorarse sin justificación.

3.2 Bun

Bun será la herramienta utilizada en el frontend para instalar dependencias y ejecutar scripts del proyecto. Su objetivo es mantener un flujo de trabajo rápido y uniforme entre los equipos.

- Instalar dependencias del frontend utilizando Bun.
- Ejecutar los scripts del frontend utilizando Bun.
- Mantener el archivo de bloqueo generado por Bun dentro del repositorio.
- No utilizar npm, yarn o pnpm dentro del frontend salvo que posteriormente se apruebe un cambio general.

3.3 Screaming Architecture (Feature-First)

El frontend se organizará por funcionalidades del negocio. La estructura de carpetas debe permitir entender qué hace Brotar con solo observar el proyecto. Por esta razón, las funcionalidades principales se agruparán dentro de features/ y no en carpetas globales separadas únicamente por tipo técnico.

```
src/
├── app/                                # Configuración global, providers y rutas
├── features/
│   ├── public/
│   │   ├── home/
│   │   ├── explore-projects/
│   │   ├── project-detail/
│   │   ├── how-it-works/
│   │   └── for-creators/
│   ├── auth/
│   │   ├── login/
│   │   ├── register/
│   │   └── recover-password/
│   ├── campaigns/
│   ├── creator/
│   ├── sponsor/
│   └── administration/
└── shared/
    ├── components/
    ├── hooks/
    ├── utils/
    ├── types/
    └── constants/
```

Cómo interpretar esta estructura. Si un componente, hook o servicio pertenece únicamente a una funcionalidad, debe permanecer dentro de esa feature. Solo los elementos realmente compartidos por distintas partes del sistema deben pasar a shared/.

Ejemplo. Una tarjeta de proyecto usada en Inicio, Explorar y otras páginas públicas puede ser compartida. En cambio, un formulario específico para crear una recompensa debe permanecer dentro de la funcionalidad correspondiente.

4. Backend

4.1 NestJS + TypeScript

El backend se desarrollará con NestJS y TypeScript. NestJS será la base para construir los módulos, controladores, servicios, validaciones e integración con infraestructura.

- Los datos de entrada deben definirse mediante DTOs.
- Las validaciones deben realizarse de forma explícita antes de ejecutar reglas de negocio.
- Las dependencias deben inyectarse en lugar de crearse directamente dentro de los casos de uso.
- Los módulos deben representar capacidades del sistema, por ejemplo: auth, users, campaigns, payments o verification.

4.2 pnpm como gestor exclusivo del backend

pnpm será el gestor de paquetes oficial del backend. Todos los comandos de instalación, ejecución y construcción del backend deberán utilizar pnpm.

- Agregar dependencias con pnpm.
- Ejecutar scripts con pnpm.
- Mantener el lockfile de pnpm dentro del repositorio.
- No mezclar npm, yarn o Bun dentro del backend.

Importante. Bun corresponde al frontend y pnpm al backend. Esta separación debe respetarse para evitar lockfiles duplicados y diferencias de dependencias entre integrantes.

4.3 Arquitectura Hexagonal (Ports and Adapters)

El backend utilizará Arquitectura Hexagonal. La idea principal es que las reglas de negocio no dependan directamente de PostgreSQL, del ORM, de NestJS o de un proveedor externo. El núcleo debe poder entenderse y probarse sin conocer los detalles de infraestructura.

```
src/
├── campaigns/
│   ├── domain/
│   │   ├── entities/
│   │   ├── value-objects/
│   │   └── rules/
│   ├── application/
│   │   ├── use-cases/
│   │   ├── ports/
│   │   └── dto/
│   └── infrastructure/
│       ├── controllers/
│       ├── persistence/
│       └── adapters/
├── auth/
├── users/
├── payments/
├── verification/
└── shared/
```

4.4 Responsabilidad de cada capa

Capa	Responsabilidad	Qué no debería contener
Domain	Entidades, reglas y conceptos propios del negocio.	Controladores, ORM, consultas SQL o detalles HTTP.
Application	Casos de uso y puertos/interfaces que describen lo que el sistema necesita.	Detalles específicos de PostgreSQL o proveedores externos.
Infrastructure	Controladores NestJS, repositorios, ORM, almacenamiento y adaptadores externos.	Reglas centrales del negocio que deberían vivir en Domain/Application.

Flujo simplificado. Un controlador recibe la petición, llama a un caso de uso, el caso de uso trabaja con interfaces (ports) y la infraestructura implementa esas interfaces para acceder a PostgreSQL u otros servicios.

HTTP → Controller → Use Case → Port → Adapter / Repository → PostgreSQL

Aplicación práctica para estudiantes. No es necesario crear una gran cantidad de interfaces o clases vacías. La arquitectura debe ayudar a separar responsabilidades, no convertir tareas simples en código innecesariamente complejo.

5. Base de datos

5.1 PostgreSQL

PostgreSQL será el motor de base de datos oficial. Brotar maneja información fuertemente relacionada (usuarios, campañas, aportes, pagos, verificaciones, revisiones y estados), por lo que se utilizará un modelo relacional.

- El lenguaje de consulta de la base de datos será SQL.
- Las relaciones deben representarse mediante claves primarias y foráneas.
- Las operaciones que deban mantenerse consistentes deberán aprovechar transacciones.
- Se podrá utilizar JSONB únicamente cuando una estructura realmente sea variable; no debe reemplazar relaciones normales sin necesidad.
- Los cambios de estructura de la base de datos deberán registrarse mediante migraciones y mantenerse en Git.

5.2 ORM / acceso a datos

El acceso a PostgreSQL se realizará mediante un ORM tipado. La alternativa definitiva entre Prisma y TypeORM podrá cerrarse antes de comenzar el desarrollo de persistencia; ambos deben mantenerse confinados dentro de Infrastructure.

Opción	Uso	Regla común
Prisma	ORM tipado con esquema y cliente generado.	No exponer tipos del ORM como entidades de dominio.
TypeORM	ORM integrado habitualmente con NestJS y basado en entidades/repositorios.	No colocar reglas de negocio dentro de entidades de persistencia.
Decisión pendiente dentro del stack. PostgreSQL ya está definido. La elección final entre Prisma y TypeORM puede mantenerse abierta hasta que los líderes confirmen una sola opción para todos los equipos.		

6. Contratos y comunicación entre frontend y backend

Al utilizar TypeScript en ambos lados, los equipos podrán definir estructuras de datos claras para las solicitudes y respuestas. Sin embargo, compartir tipos no significa que frontend y backend deban quedar fuertemente acoplados.

- Definir nombres y estructuras consistentes para los payloads.
- Mantener tipos compartidos únicamente para contratos realmente comunes.
- No importar código interno del backend dentro del frontend.
- No reutilizar directamente entidades de base de datos como modelos de interfaz.
- Cuando se modifique un contrato, actualizar de forma coordinada frontend y backend.

```
Frontend
  ↓ request / response tipado
API Backend
  ↓
Application / Domain
  ↓
Infrastructure
  ↓
PostgreSQL
```

7. Directrices prácticas para todos los equipos

7.1 Mantener una única forma de trabajar

- Frontend: React + TypeScript + Bun.
- Backend: NestJS + TypeScript + pnpm.
- Base de datos: PostgreSQL.
- Frontend organizado por features.
- Backend organizado por módulos y capas hexagonales.
- No cambiar el stack de forma individual sin aprobación de los responsables del proyecto.

7.2 Evitar complejidad prematura

El hecho de utilizar buenas arquitecturas no significa que el MVP necesite todas las técnicas avanzadas desde el inicio. No se deben introducir tecnologías adicionales solo por parecer más profesionales.

- No crear microservicios si el sistema todavía puede resolverse como una aplicación backend organizada por módulos.
- No introducir colas, brokers, cachés distribuidas o servicios adicionales sin una necesidad real.
- No crear abstracciones genéricas antes de tener al menos un caso concreto que las necesite.
- No duplicar lógica entre features o módulos; reutilizar cuando realmente sea compartida.
- Priorizar primero que el flujo funcione correctamente y después optimizar.

7.3 Convenciones básicas de repositorio

- Cada equipo debe mantener su proyecto ejecutable desde un repositorio limpio.
- Los archivos .env reales no deben subirse; debe existir un .env.example cuando se utilicen variables de entorno.
- Los lockfiles correspondientes deben versionarse: Bun en frontend y pnpm en backend.
- Las migraciones de base de datos deben formar parte del repositorio.
- El README debe indicar cómo instalar y ejecutar frontend, backend y base de datos cuando estas capas existan.
- Los commits deben ser claros y describir el cambio realizado.

8. Beneficios esperados

Decisión	Beneficio
TypeScript end-to-end	Reduce errores de tipos y facilita comprender qué datos circulan entre las capas.
React + Screaming Architecture	Permite encontrar rápidamente las funcionalidades y evita un frontend dividido solo por tipos técnicos.
NestJS + Arquitectura Hexagonal	Separa reglas de negocio de frameworks, base de datos y servicios externos.
Bun en frontend	Unifica la gestión y ejecución del frontend con una herramienta rápida.
pnpm en backend	Mantiene una gestión consistente de dependencias para NestJS.
PostgreSQL	Aporta un modelo relacional sólido para información conectada y operaciones transaccionales.
ORM aislado en Infrastructure	Permite cambiar detalles de persistencia sin contaminar el dominio.

9. Checklist rápido antes de desarrollar

- ¿La funcionalidad pertenece a una feature o módulo concreto?
- ¿El código específico está dentro de esa feature/módulo y no en shared sin necesidad?
- ¿Los datos están tipados y no dependen de any sin justificación?

- ¿En backend la regla de negocio está separada de PostgreSQL y del ORM?
- ¿Prisma o TypeORM se utilizan únicamente desde Infrastructure?
- ¿Frontend usa Bun y backend usa pnpm sin mezclar gestores?
- ¿La solución es la más simple que cumple la necesidad actual?

10. Stack tecnológico oficial

Frontend: React + TypeScript, Bun y Screaming Architecture (Feature-First).

Backend: NestJS + TypeScript, pnpm y Arquitectura Hexagonal (Ports and Adapters).

Base de datos: PostgreSQL utilizando SQL.

Acceso a datos: Prisma o TypeORM, ubicado únicamente en la capa de Infrastructure.

Brotar · Estándares tecnológicos y arquitectónicos